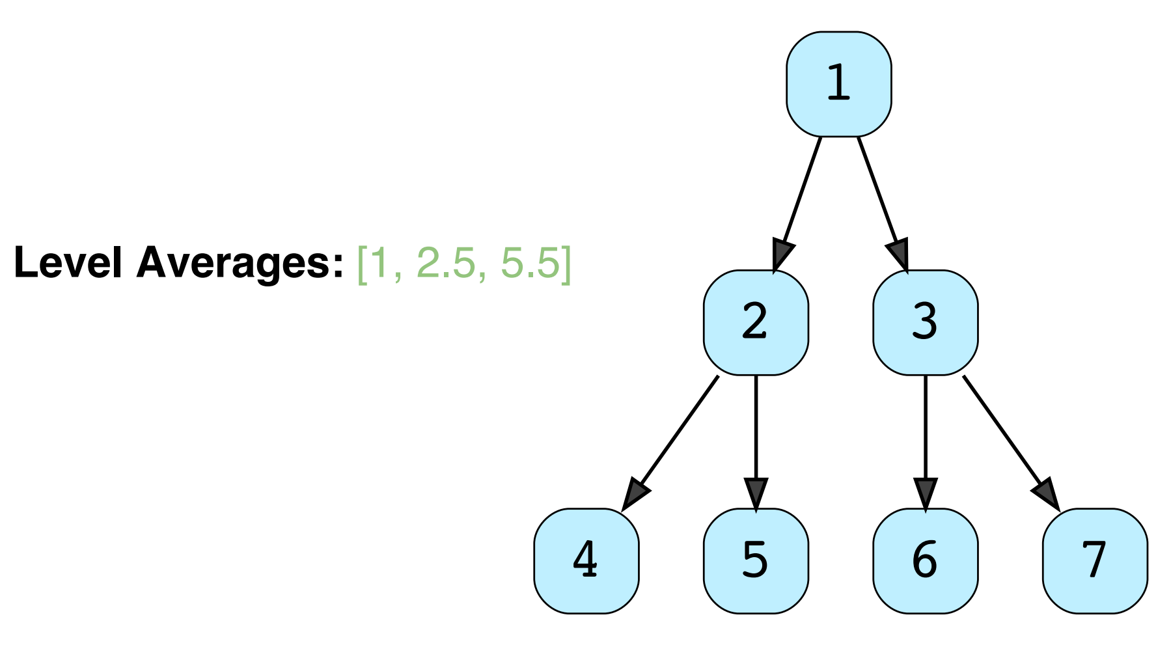
Solution: Level Averages in a Binary Tree

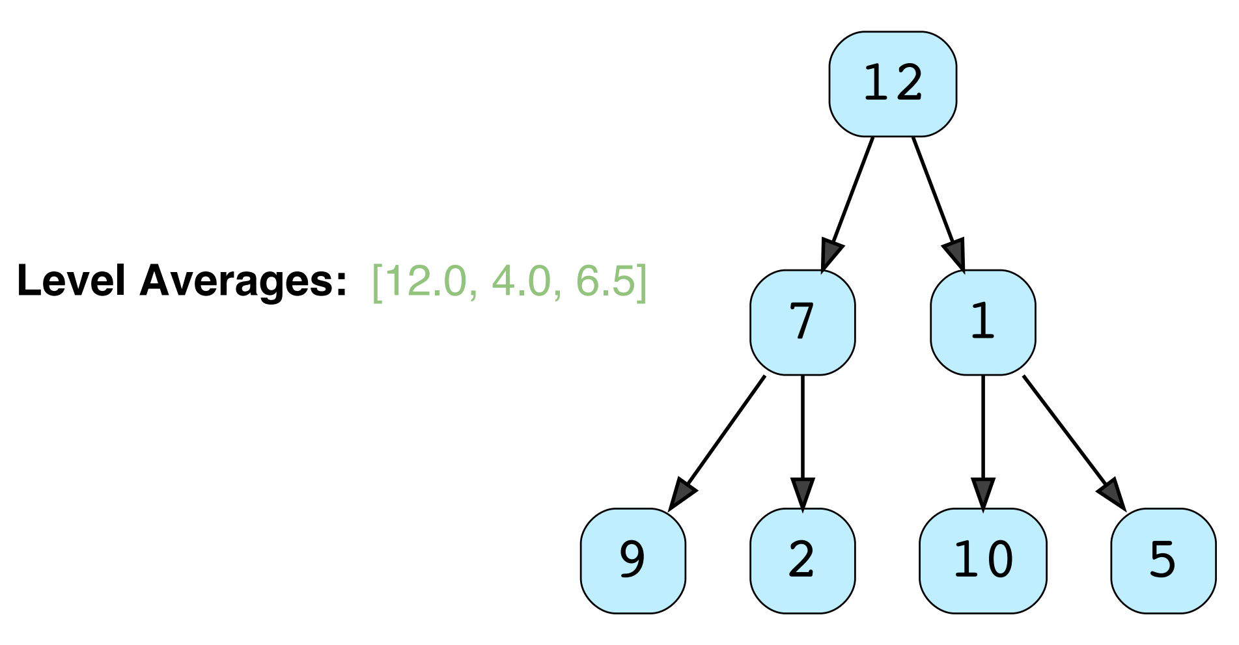
Problem Statement

Given a binary tree, populate an array to represent the **averages of all of its levels**.

Example 1:



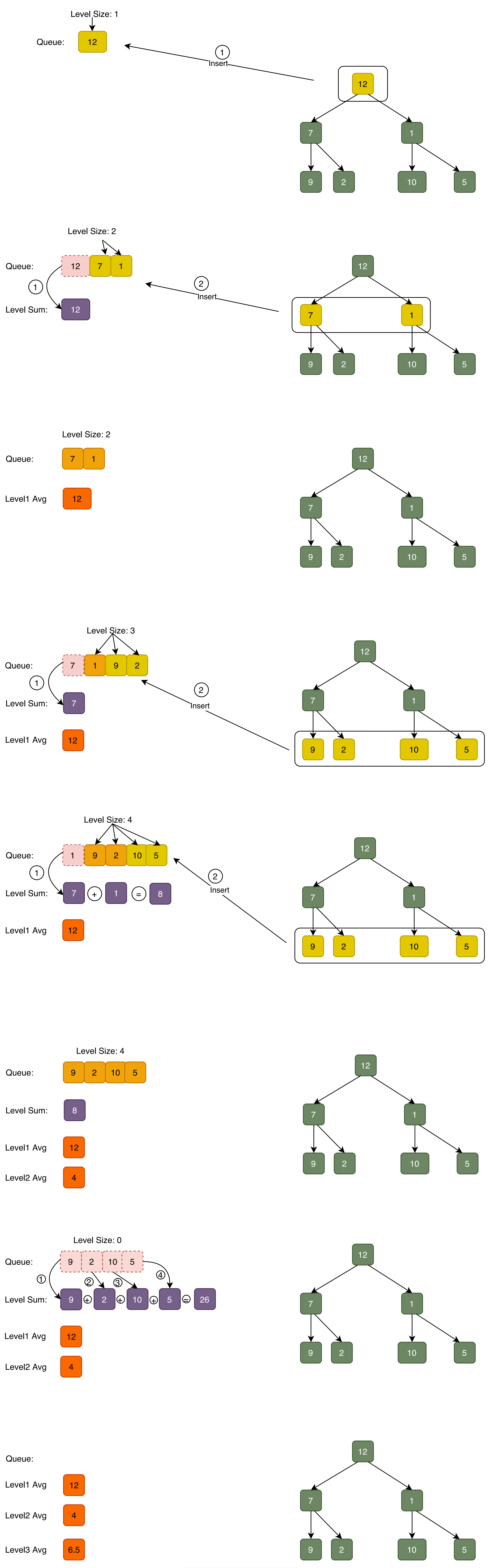
Example 2:



Solution

This problem follows the **Binary Tree Level Order Traversal** pattern. We can follow the same **BFS** approach. The only difference will be that instead of keeping track of all nodes of a level, we will only track the running sum of the values of all nodes in each level. In the end, we will append the average of the current level to the result array.

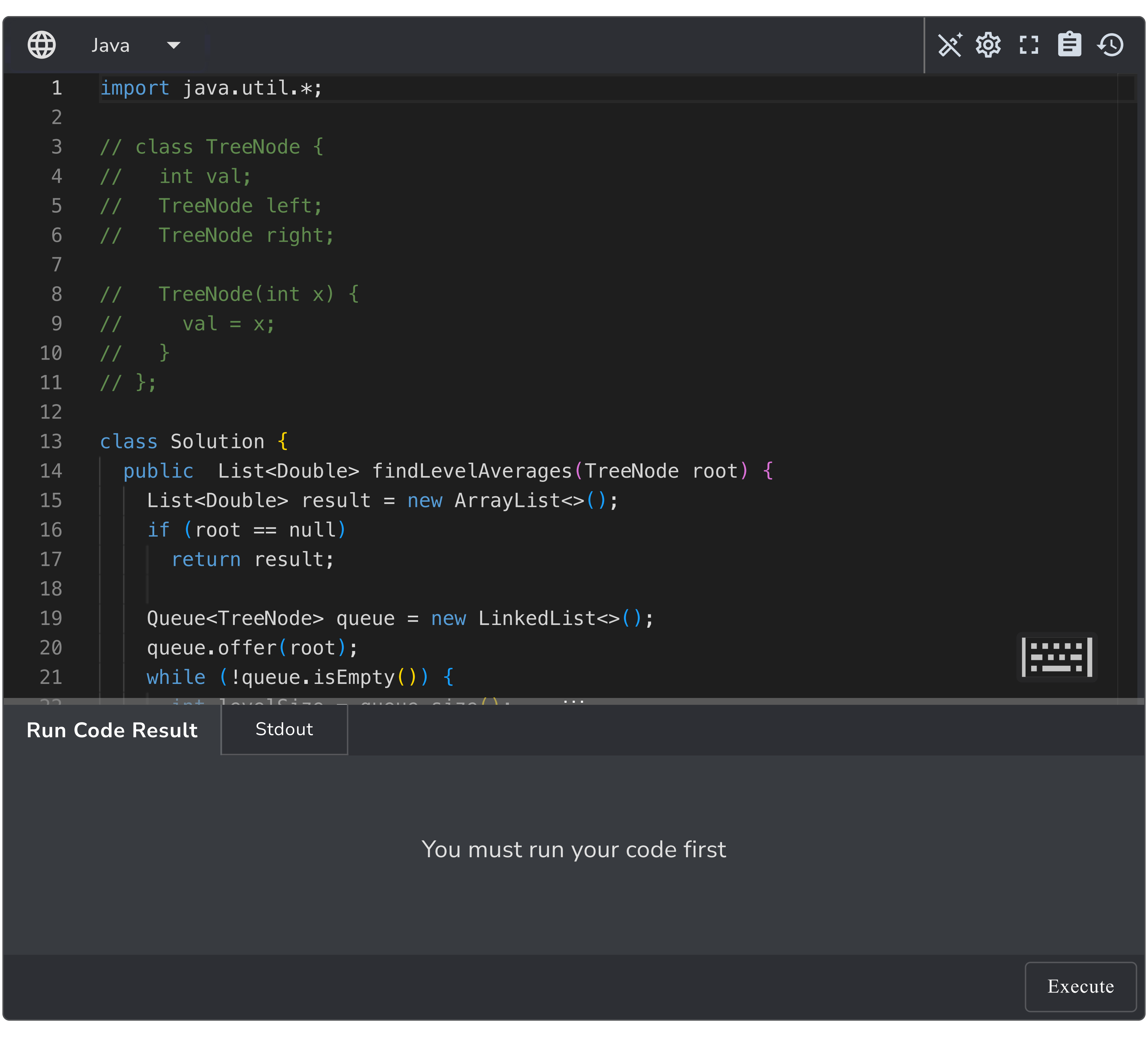
Here is the visual representation of the algorithm:



Level Averages in a BT

Code

Here is the code for this algorithm:



```java
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {
  public  List<Double> findLevelAverages(TreeNode root) {
    List<Double> result = new ArrayList<>();
    if (root == null)
      return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
```

You must run your code first

Time Complexity

The time complexity of the above algorithm is $O(N)$, where N is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space Complexity

The space complexity of the above algorithm will be $O(N)$ which is required for the queue. Since we can have a maximum of $N/2$ nodes at any level (this could happen only at the lowest level), therefore we will need $O(N)$ space to store them in the queue.

Similar Problems

Problem 1: Find the largest value on each level of a binary tree.

Solution: We will follow a similar approach, but instead of having a running sum we will track the maximum value of each level.

```
maxValue = max(maxValue, currentNode.val)
```

← Previous
Level Averages In A Binary Tree
(Easy)

Next →
Minimum Depth Of A Binary Tree
(Easy)

☐ Mark as Completed